

PROJET DE TECHNIQUE DE PROGRAMMATION

StéganoCrypt - Chiffrement & Stéganographie

Licence 1 - Département Informatique - UPPA

0. Présentation du projet

Ce projet vous demande de concevoir et implémenter en langage C un outil en ligne de commande baptisé StéganoCrypt. Cet outil combine deux domaines :

- **Cryptographie** - La cryptographie symétrique : chiffrement de messages texte par les algorithmes Transposition par colonnes.
- **Stéganographie** - La stéganographie : dissimulation d'un message (chiffré ou non) à l'intérieur d'une image numérique.
- **Traitement d'image** - Le traitement d'image : lecture, écriture et transformations géométriques (rotation, symétrie, redimensionnement) d'images PPM.

Le projet est à rendre au plus tard le 10 mai à 23h59 sur e-learn.

1. Contexte & scénario

Alice souhaite envoyer un message secret à Bob sans éveiller les soupçons. Pour cela, elle :

1. Saisit un message texte.
2. Le chiffre avec l'algorithme de son choix (Transposition par colonnes).
3. Dissimule le message chiffré dans une image banale (photo de vacances, paysage...) en modifiant de façon imperceptible les bits de poids faible de chaque pixel.
4. Envoie l'image à Bob. Bob utilise le même programme pour extraire et déchiffrer le message.

Bob, de son côté :

5. Charge l'image reçue.
6. Extrait le message caché.
7. Déchiffre le message avec la même clé.

2. Architecture du programme

Le programme sera organisé en plusieurs modules indépendants. Chaque module correspond à un fichier .c et son en-tête .h :

Fichier	Module	Rôle
crypto.c / .h	Cryptographie	Chiffrement et déchiffrement par transposition par colonnes
image.c / .h	Images	Structures sPixel, sImagePPM, sImagePGM, sImagePBM ; chargement et sauvegarde
steganographie.c / .h	Stéganographie	Dissimulation et extraction de messages dans les pixels
traitement.c / .h	Traitement	Conversion niveaux de gris, seuillage, convolution (bonus)
main.c	Programme principal	Parsing des arguments, orchestration des modules

3. Fonctionnalités demandées

3.1 Module Cryptographie

Ce module implémente un seul algorithme de chiffrement, nouveau par rapport aux TP : la transposition par colonnes. Les deux fonctions suivantes sont obligatoires :

```
char* chiffrer_transposition(const char* message, const int* cle, int taille_cle);
char* dechiffrer_transposition(const char* message, const int* cle, int taille_cle);
```

Algorithme — Transposition par colonnes

Le message est écrit ligne par ligne dans une grille de N colonnes (N = taille de la clé), puis lu colonne par colonne dans l'ordre défini par la clé numérique.

Exemple avec la clé {3, 1, 4, 2} (4 colonnes) et le message "CRYPTOGRAPHIE" :

```
      col 0  col 1  col 2  col 3
Ordre clé :   3     1     4     2
ligne 0  :   C     R     Y     P
ligne 1  :   T     O     G     R
ligne 2  :   A     P     H     I
ligne 3  :   E     *     *     *      (* = padding)

Lecture dans l'ordre de la clé triée (col 1, col 3, col 0, col 2) :
col 1 : R O P  → "ROP"
col 3 : P R I  → "PRI"
col 0 : C T A E → "CTAE"
col 2 : Y G H  → "YGH"
Chiffré : "ROPRICTAEYGH"
```

Si la longueur du message n'est pas un multiple de N, on complète la dernière ligne avec des espaces (padding). Le déchiffrement inverse l'ordre des colonnes pour reconstituer la grille, puis lit ligne par ligne.

- La fonction retourne un pointeur vers une chaîne allouée dynamiquement. L'appelant est responsable de libérer la mémoire (free).
- Le déchiffrement est l'inverse exact du chiffrement : `dechiffrer(chiffrer(m)) == m` (espaces de padding retirés).
- Paramètre invalide : `taille_cle ≤ 0` ou clé contenant des doublons pour la transposition → retourner NULL et écrire un message d'erreur.

3.2 Module Images

Pour ce module, vous pouvez vous référer au TP 9 sur les fichiers..

```
typedef struct { unsigned char r, g, b; } sPixel;
typedef struct { int largeur, hauteur, max; sPixel** pixels; } sImagePPM;
```

```
sImagePPM* charger_ppm(const char* chemin);
int         sauvegarder_ppm(const sImagePPM*, const char*);
void        liberer_ppm(sImagePPM* img);
```

Rotation 90° (sens horaire)

Une image de dimensions $L \times H$ devient une image $H \times L$. Le pixel (i, j) de l'image originale correspond au pixel $(j, L-1-i)$ de l'image résultat.

```
sImagePPM* rotation_90(const sImagePPM* img);
```

Symétrie axiale

Deux symétries sont demandées. Pour la symétrie horizontale (retournement gauche-droite) : le pixel (i, j) va en $(i, L-1-j)$. Pour la symétrie verticale (retournement haut-bas) : le pixel (i, j) va en $(H-1-i, j)$.

```
sImagePPM* symetrie_h(const sImagePPM* img); // axe vertical
sImagePPM* symetrie_v(const sImagePPM* img); // axe horizontal
```

Redimensionnement (nearest neighbour)

L'image est redimensionnée à une nouvelle largeur L' et une nouvelle hauteur H' . Pour chaque pixel (i, j) de l'image résultat, on calcule le pixel source le plus proche :

```
src_i = (int)(i * H / H')
src_j = (int)(j * L / L')
sImagePPM* redimensionner(const sImagePPM* img, int nouv_largeur, int nouv_hauteur);
```

Point de vigilance : toutes ces fonctions allouent et retournent une nouvelle image. L'image originale n'est jamais modifiée. L'appelant libère les deux images séparément.

Toutes les allocations mémoire doivent être vérifiées. En cas d'erreur, la fonction retourne NULL.

3.3 Module Stéganographie (LSB)

Le module de stéganographie permet de cacher un message texte à l'intérieur d'une image PPM existante. Le message n'est pas converti en image, il est dissimulé dans les pixels en modifiant chaque composante de façon imperceptible.

Règle fondamentale : on ne modifie que le bit de poids faible (bit 0, appelé LSB) de chaque composante R, G ou B. Cela change la valeur de ± 1 au maximum, invisible à l'œil humain.

Étape 1 — le message chiffré est une chaîne de caractères ordinaire

Après chiffrement, le message reste une chaîne C (char*). Par exemple, « HI » chiffré devient « NQ ». C'est du texte ordinaire, avec des codes ASCII comme n'importe quel caractère :

'N' = code ASCII 78 | 'Q' = code ASCII 81

En C, un char EST déjà un entier. message[0] vaut 78, message[1] vaut 81. Pas besoin de conversion.

Étape 2 — décomposer chaque code ASCII en 8 bits

On décompose chaque entier en 8 bits par divisions successives par 2. Exemple sur « N » (78) :

```
val = 78
bit 0 : 78 % 2 = 0, val = 78 / 2 = 39
bit 1 : 39 % 2 = 1, val = 39 / 2 = 19
bit 2 : 19 % 2 = 1, val = 19 / 2 = 9
bit 3 : 9 % 2 = 1, val = 9 / 2 = 4
bit 4 : 4 % 2 = 0, val = 4 / 2 = 2
bit 5 : 2 % 2 = 0, val = 2 / 2 = 1
bit 6 : 1 % 2 = 1, val = 1 / 2 = 0
bit 7 : 0 % 2 = 0
```

On obtient les bits 0,1,1,1,0,0,1,0 — soit 01001110 — ce qui correspond bien à 78 en binaire.

Étape 3 — cacher chaque bit dans le LSB d'une composante

Chaque bit du message va être mis dans le bit 0 d'une composante de l'image. Exemple concret sur le pixel 10 de l'image (les 3 premiers bits de « N ») :

Avant : R=200 (LSB=0) G=147 (LSB=1) B=89 (LSB=1)

Bits à cacher : bit0=0, bit1=1, bit2=1 (premiers bits de 'N'=78)

Après : R=200 (LSB forcé à 0, inchangé) G=146 (LSB forcé à 1→146+0=146... non : (147/2)*2+1=147) B=88 (LSB forcé à 1)

La variation est de 0 ou 1 au maximum sur chaque composante. Sur 255 niveaux de couleur possibles, cela est totalement invisible.

Étape 4 — protocole complet d'encodage dans l'image

Les composantes de l'image sont parcourues de gauche à droite, ligne par ligne. L'encodage se fait en deux phases successives :

8. Phase L — positions 0 à 31 : on encode la longueur du message (strlen) sur 32 bits. Lors de l'extraction, ces 32 bits seront lus en premier pour savoir combien de caractères récupérer.
9. Phase M — positions 32 et suivantes : on encode chaque caractère du message chiffré sur 8 bits (son code ASCII). Le caractère d'indice c occupe les positions 32 + c×8 à 32 + c×8 + 7.

Exemple : pour le message chiffré « NQ » (2 caractères), on modifie $32 + 2 \times 8 = 48$ composantes de l'image au total. Sur une image 100×100 , cela représente 48 composantes sur 30 000, moins de 0,2 % de l'image.

Le code ASCII en pratique

Chaque caractère affichable a un code ASCII fixé par le standard. En C, ce code est directement accessible via le type char :

- 'A' = 65, 'B' = 66, ..., 'Z' = 90
- 'a' = 97, 'b' = 98, ..., 'z' = 122
- '0' = 48, '1' = 49, ..., '9' = 57
- ' ' (espace) = 32

Un message chiffré peut contenir n'importe quel caractère de la table ASCII (codes 0 à 127). Chacun tient sur 8 bits. Les fonctions `get_bit` et `set_lsb` fonctionnent de façon identique quel que soit le caractère.

Décomposition en 6 fonctions

Le module `steganographie.c` utilise le type `sImagePPM` défini dans `image.h`. La décomposition suivante est obligatoire.

Règle d'or : chaque fonction fait UNE seule chose. Division entière (/) et modulo (%) suffisent pour tout.

Fonction 1 — extraire le bit de rang k d'un entier :

```
int get_bit(int valeur, int k);
```

Diviser valeur par 2 exactement k fois, puis retourner le reste par 2. Fonctionne sur un code ASCII comme sur la longueur.

Fonction 2 — forcer le LSB d'une composante à une valeur donnée :

```
unsigned char set_lsb(unsigned char composante, int bit);
```

(composante / 2) * 2 remet le LSB à zéro, puis on ajoute le bit voulu.

Exemple : `set_lsb(200, 1)` retourne 201. `set_lsb(201, 0)` retourne 200.

Fonction 3 — écrire un bit dans la n-ième composante de l'image :

```
void encoder_bit(sImagePPM* img, int n, int bit);
```

La n-ième composante est la composante numéro (n % 3) du pixel numéro (n / 3), situé ligne (n/3 / largeur) et colonne (n/3 % largeur). On modifie cette composante avec `set_lsb`.

Fonction 4 — lire le LSB de la n-ième composante :

```
int decoder_bit(const sImagePPM* img, int n);
```

Même calcul d'indices qu'`encoder_bit`. On lit la composante et on retourne `get_bit(composante, 0)`. C'est l'inverse exact d'`encoder_bit`.

Fonction 5 — cacher un message dans une image :

```
int cacher_message(sImagePPM* img, const char* message);
```

Retourne 0 si succès, -1 si le message est trop long.

10. Calculer `lon = strlen(message)`. Vérifier : $(32 + lon * 8) \leq \text{largeur} * \text{hauteur} * 3$. Sinon retourner -1.

11. Encoder la longueur : pour k de 0 à 31, appeler `encoder_bit(img, k, get_bit(lon, k))`.
12. Encoder le message : pour chaque caractère c et chaque bit j de 0 à 7, appeler `encoder_bit(img, 32 + c*8 + j, get_bit(message[c], j))`.

Fonction 6 — extraire un message caché depuis une image :

```
char* extraire_message(const sImagePPM* img);
```

Retourne une chaîne allouée avec malloc (à libérer avec free), ou NULL en cas d'erreur.

13. Reconstruire lon depuis les 32 premiers bits : lon = 0, puis pour k de 0 à 31 : lon += decoder_bit(img, k) * puissance(2, k).
14. Vérifier que lon est cohérente (lon > 0 et (32 + lon*8) <= largeur * hauteur * 3). Sinon retourner NULL.
15. Allouer msg = malloc(lon + 1). Reconstruire chaque caractère sur 8 bits. Poser msg[lon] = '\0'.

Vérification : après `cacher_message(img, msg)`, un appel à `extraire_message(img)` doit redonner exactement msg. Testez avec `assert(strcmp(msg, extrait) == 0)`.

3.4 Module Traitement d'image

Ce module regroupe les transformations géométriques définies en section 3.2, ainsi que les deux fonctions de traitement colorimétrique ci-dessous. Ces dernières sont plus simples mais nécessaires pour compléter la chaîne de traitement :

```
sImagePGM* convertir_gris(const sImagePPM* img);
sImagePGM* seuiller(const sImagePGM* img, int seuil);
```

Pour l'option -rot, la rotation est appliquée autant de fois que nécessaire (ex. -rot 180 = deux appels à rotation_90). Pour -resize, les dimensions cibles sont passées en arguments.

3.5 Programme principal — menu interactif

Le programme s'utilise avec un menu console. Les paramètres (chemin du fichier, message, clé...) peuvent être passés en argument sur la ligne de commande pour éviter de les retaper à chaque exécution. Si un paramètre n'est pas fourni, le programme le demande via scanf.

Principe : arguments optionnels

Le main() reçoit argc et argv. Selon le nombre d'arguments passés, il pré-remplit les variables avant d'afficher le menu :

```
./steganocrypt photo.ppm secret.ppm "Rendez-vous" 4 3 1 4 2
               ^           ^           ^           ^ ^
               source      sortie      message  taille  clé
```

```
./steganocrypt photo.ppm          /* seul le fichier source fourni */
./steganocrypt                    /* rien : tout sera demandé */
```

Dans tous les cas, le menu s'affiche ensuite normalement. Les arguments servent uniquement à pré-remplir les variables, pas à court-circuiter le menu.

Règle simple : si argv[i] existe, on l'utilise. Sinon, on demande avec printf + scanf. Le menu reste toujours le point d'entrée.

Structure du main()

Le main() lit les arguments disponibles, les stocke dans des variables globales, puis lance le menu :

```
/* Variables globales partagées par tous les sous-menus */
char g_source[256] = "";
char g_sortie[256] = "";
char g_message[256] = "";
int g_cle[10];
int g_taille_cle = 0;
sImagePPM* g_img = NULL;
```

La fonction menu_principal() tourne en boucle avec un while(1) jusqu'au choix "Quitter". Elle affiche l'état courant (nom du fichier chargé, message en cours) pour que l'étudiant sache où il en est :

```
void menu_principal(void) {
    int choix;
    while (1) {
        printf("\n=== StéganoCrypt ===");
        printf(" | fichier: %s", strlen(g_source)>0 ? g_source : "(aucun)");
        printf(" | message: %s\n", strlen(g_message)>0 ? g_message : "(aucun)");
        printf("1. Cryptographie\n");
        printf("2. Images\n");
        printf("3. Stéganographie\n");
        printf("4. Traitement et compression\n");
        printf("0. Quitter\n");
        printf("Votre choix : ");
        scanf("%d", &choix);
        if (choix == 1) menu_cryptographie();
        else if (choix == 2) menu_images();
        else if (choix == 3) menu_steganographie();
        else if (choix == 4) menu_traitement();
        else if (choix == 0) { printf("Au revoir !\n"); return; }
        else printf("Choix invalide.\n");
    }
}
```

Chaque sous-menu suit le même schéma : boucle while(1), affichage des options, lecture du choix, appel de la fonction correspondante, 0 pour revenir au menu principal.

Principe commun des sous-menus

Chaque sous-menu utilise une fonction utilitaire demander_si_vide() qui utilise la variable globale si elle est déjà remplie, ou la demande sinon. Cela évite de répéter la même logique partout :

Exemple d'utilisation : demander_si_vide(g_source, "Fichier source : "); — si g_source a été passé en argument, la question s'affiche avec la valeur déjà remplie et l'utilisateur n'a rien à taper.

Sous-menu 1 — Cryptographie

Ce sous-menu permet de tester les deux fonctions de chiffrement et déchiffrement, ainsi qu'un test automatique qui vérifie que déchiffrer(chiffrer(m)) == m.

```
--- Cryptographie ---
1. Chiffrer un message
```

```
2. Déchiffrer un message
3. Test automatique (chiffrer puis déchiffrer)
0. Retour
```

Pour l'option 1, le programme demande le message et la clé :

L'option 3 (test automatique) enchaîne chiffrement et déchiffrement et affiche OK ou ERREUR :

```
char* c = chiffrer_transposition(g_message, g_cle, g_taille_cle);
char* r = dechiffrer_transposition(c, g_cle, g_taille_cle);
if (strcmp(g_message, r) == 0) printf("TEST OK\n");
else printf("TEST ERREUR : '%s' != '%s'\n", g_message, r);
free(c); free(r);
```

Sous-menu 2 — Images

Ce sous-menu permet de charger une image et d'afficher ses informations, ce qui est essentiel pour vérifier que le chargement fonctionne avant de l'utiliser dans les autres modules.

```
--- Images ---
1. Charger une image PPM et afficher ses infos
2. Charger une image PGM et afficher ses infos
3. Sauvegarder l'image courante
0. Retour
```

Pour l'option 1, le programme demande le nom du fichier, charge l'image avec `charger_ppm()` et affiche la largeur, la hauteur et la valeur max. L'image reste en mémoire pour être utilisée par les autres sous-menus.

Sous-menu 3 — Stéganographie

Ce sous-menu permet de cacher un message dans l'image courante, de l'extraire, et de tester les deux opérations à la suite.

```
--- Stéganographie ---
1. Cacher un message dans l'image courante
2. Extraire le message de l'image courante
3. Test automatique (cacher puis extraire)
0. Retour
```

L'option 3 vérifie le cycle complet sur l'image courante :

Sous-menu 4 — Traitement et compression

Ce sous-menu regroupe toutes les transformations géométriques et la compression. Chaque option remplace l'image courante par le résultat de la transformation.

```
--- Traitement et compression ---
1. Rotation 90° (sens horaire)
2. Symétrie horizontale (gauche-droite)
3. Symétrie verticale (haut-bas)
4. Redimensionner
5. Compresser (blocs de n pixels)
6. Convertir en niveaux de gris
7. Seuillage
0. Retour
```

Pour l'option 1, le programme applique `rotation_90()` sur l'image courante et remplace l'image :

Pour l'option 5 (compression), le programme demande la valeur de n :

Rappel : les fonctions de transformation retournent une NOUVELLE image. Il faut toujours libérer l'ancienne avec `liberer_ppm()` avant de remplacer `img_courante`, sinon la mémoire fuit.

Astuce : après chaque transformation, proposez de sauvegarder avec l'option 4 du sous-menu Images. Ouvrez le fichier PPM dans un visualiseur (GIMP, IrfanView, etc.) pour voir le résultat visuellement.

4. Fonctionnalités bonus (facultatif)

Les bonus ne sont pas obligatoires mais permettent d'obtenir des points supplémentaires. Chaque bonus doit fonctionner correctement pour être comptabilisé.

- **B1** — Convolution : implémenter `produitConvolution` et l'option `-filtre <fichier_noyau>` (cf. TP Fichiers, section 2).
- **B2** — Stéganographie PGM : adapter `cache_message/extraire_message` pour les images en niveaux de gris.
- **B3** — Statistiques : afficher un histogramme ASCII des fréquences de lettres dans le message chiffré.

5. Modalités de rendu

L'archive ZIP doit respecter l'arborescence suivante :

```
NOM_Prenom_steganocrypt/  
├── src/  
│   ├── main.c  
│   ├── crypto.c      crypto.h  
│   ├── image.c       image.h  
│   ├── steganographie.c steganographie.h  
│   └── traitement.c  traitement.h  
└── Makefile
```

Le Makefile doit permettre de compiler le projet avec la simple commande `make`. Les flags `-Wall -Wextra -std=c99` sont obligatoires.

7. Conseils & bonnes pratiques

- Testez chaque fonction indépendamment avant de l'intégrer : un bug non détecté se propage.
- Utilisez `valgrind --leak-check=full ./steganocrypt` régulièrement pour détecter les fuites.
- N'hésitez pas à créer des fonctions utilitaires (ex. `est_lettre_maj`, `indice_lettre...`).
- Gérez les cas limites : image trop petite pour le message, clé de transposition vide ou contenant des doublons, dimensions nulles pour le redimensionnement.
- Écrivez les en-têtes `.h` avec des gardes d'inclusion (`#ifndef` / `#define` / `#endif`).

Rappel : le plagiat est strictement interdit. Tout code rendu doit être le vôtre. Les similitudes entre rendus seront analysées automatiquement.